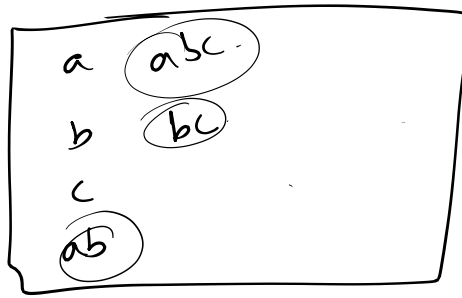


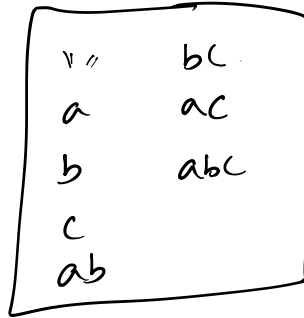
abc



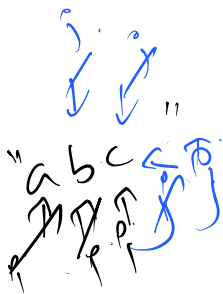
→ subsequences of "abcd"

abc

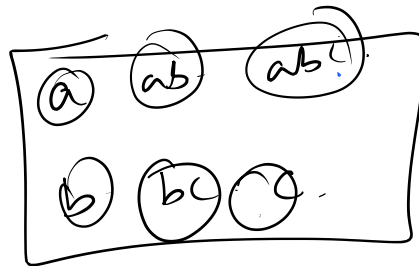
→



→ subsequences of "abc"



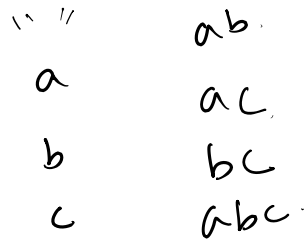
p



→ subsequences

"abc"

→



"ab"

→

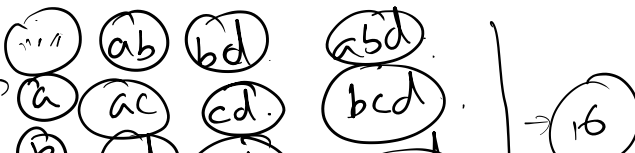


"a" →

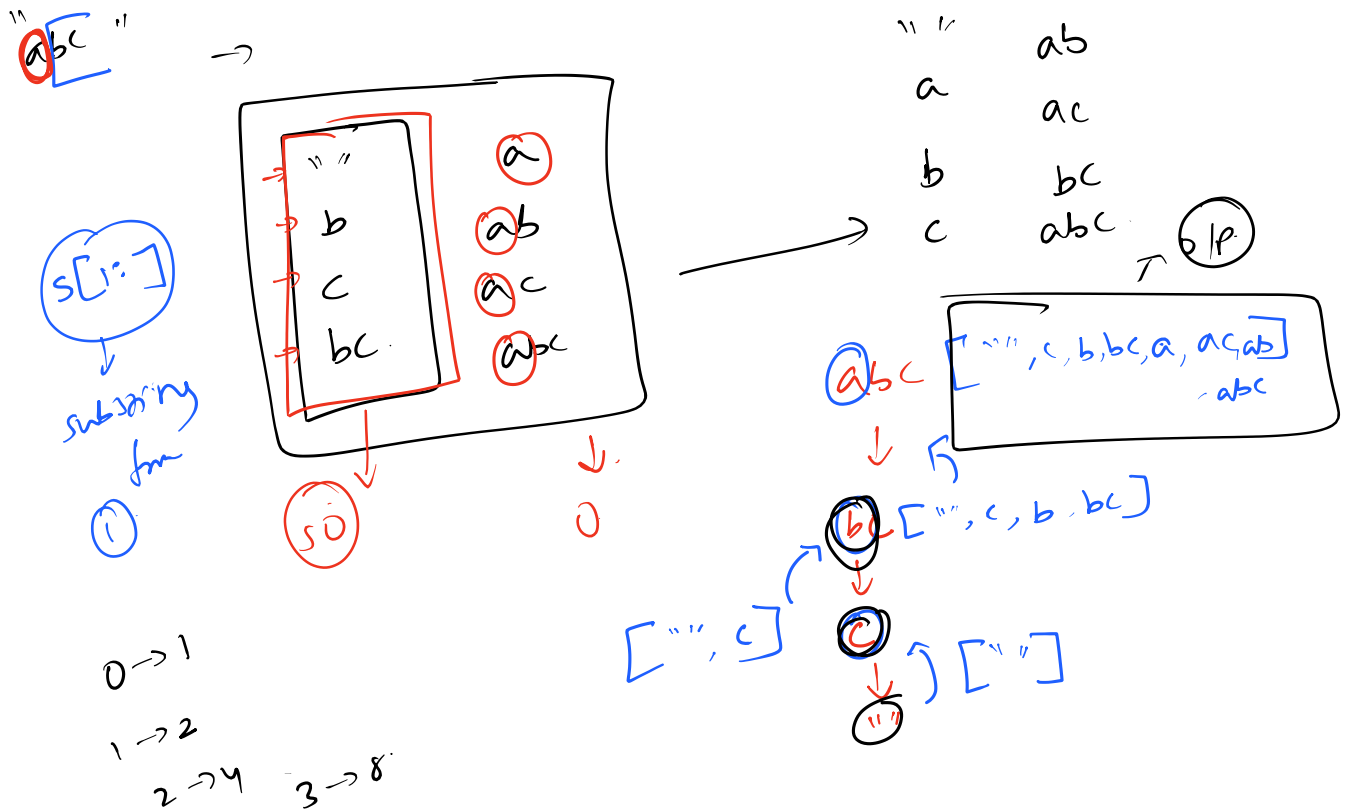
"a"

"abcd"

→



$\{ \emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\} \}$



$\{1, 2, 3\} \rightarrow$

$[\]$	$[1, 2]$
$[1]$	$[1, 3]$
$[2]$	$[2, 3]$
$[3]$	$[1, 2, 3]$

78. Subsets

Solved

Medium

Topics

Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets* (the power set).

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

$[1, 2, 3]$
 $\downarrow$
 $[\]$ $[1, 2]$
 $[1]$ $[1, 3]$
 $[2]$ $[2, 3]$
 $[3]$ $[1, 2, 3]$

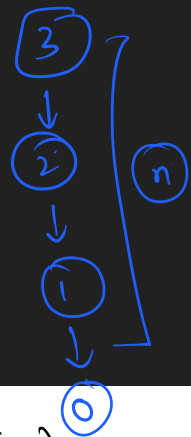
Example 1:

Input: `nums = [1,2,3]`

Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

```
class Solution:
```

```
def subsets(self, nums: List[int]) -> List[List[int]]:
    if len(nums) == 0 :
        return [[]]
    smallOutput = self.subsets(nums[1 : ])
    output = []
    for i in range(len(smallOutput)) :
        output.append(smallOutput[i])
    for subset in smallOutput :
        output.append([nums[0]] + subset)
    return output
```



$\downarrow$
T.N $\rightarrow O(2^n)$ S.C $\rightarrow O(n \cdot 2^n) \rightarrow O(2^n)$

17. Letter Combinations of a Phone Number

Medium

Topics

Companies

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



23 abc
def

ad ae af
bd be bf
cd ce cf

2 3 4

abc def ghi

adg	aeg	afg	bdg	beg	bfh
adh	aei	afh	bdi	bei	bfi
adi	aei	afi			
cdg	ceg	cfg			
cdh	ceh	cfh			
cdi	cei	cfi			

2 3

"def"

ad	bd	cd
ae	be	ce
af	bf	cf

"def"

[ad, ae, af, bd, be, bf, cd, ce, cf]
 [d, e, f]
 2 3
 ↓
 3
 ↓
 3
 ↓
 3

"abc"
 999
 3 x 4 x 4 → 64
 7
 "def"
 4 x 4 x 4 → 64

2 3 2 3 4
 3 x 3 → 9
 3 x 3 x 3
 27

"abc" 2 3
 ↓
 3
 ↑ "def" → [d, e, f]
 []
 ↓
 3

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        map = {
            "2": "abc",
            "3": "def"
```

→ pNo: 17

27


```

    "3" : "def",
    "4" : "ghi",
    "5" : "jkl",
    "6" : "mno",
    "7" : "pqrs",
    "8" : "tuv",
    "9" : "wxyz"
}

def helper(digits):
    if len(digits) == 0:
        return [""]
    smallOutput = helper(digits[1:])
    options = map[digits[0]]
    output = []
    for i in range(len(options)):
        for j in range(len(smallOutput)):
            output.append(options[i] + smallOutput[j])
    return output

return helper(digits)

```

$\rightarrow T.C \rightarrow O(4^n) \rightarrow (n^{*4})$
 $\rightarrow S.C \rightarrow O(4^n) \rightarrow (n^{*4})$

2 3 4 5 7 9 } 2 5 7
 —————→
 (n)

22

5, 5
 ↓
 4, 5
 ↓
 3, 20
 ↓
 2, 60
 ↓
 1, 120
 ↓
 0, 120

ans []

120

```

def printFact(n, ansSoFar):
    if n == 0:
        print(ansSoFar)
        return
    ansSoFar = ansSoFar * n
    print(n, ansSoFar)
    printFact(n - 1, ansSoFar)

```

0	1	2	3
*	/	+	-

0 1 2 3

4 5 6 7

8

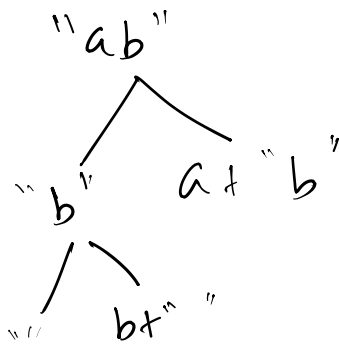
1 % len(nums)

```
printFact(5, 1)
```

"abc" →

" "	ab
a	ac
b	bc
c	abc

```
printSubsequences(s, outputSoFar):
    if len(s) == 0:
        print(outputSoFar)
    return
```



```
def printSubsequences(s, outputSoFar):
    if len(s) == 0:
        print(outputSoFar)
        return
    printSubsequences(s[1:], outputSoFar)
    printSubsequences(s[1:], outputSoFar + s[0])

printSubsequences("abc", "")
```

141. Linked List Cycle

Attempted 🕒

Easy

📁 Topics

🏢 Companies

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list

that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` if there is a cycle in the linked list. Otherwise, return `false`.

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = head
        fast = head
        while fast is not None and fast.next is not None :
            fast = fast.next.next
            slow = slow.next
            if slow == fast :
                return True
        return False
```

999. Available Captures for Rook

Solved ✓

Easy

Topics

Companies

You are given an `8 x 8` **matrix** representing a chessboard. There is **exactly one** white rook represented by `'R'`, some number of white bishops `'B'`, and some number of black pawns `'p'`. Empty squares are represented by `'.'`.

A rook can move any number of squares horizontally or vertically (up, down, left, right) until it reaches another piece or the edge of the board. A rook is **attacking** a pawn if it can move to the pawn's square in one move.

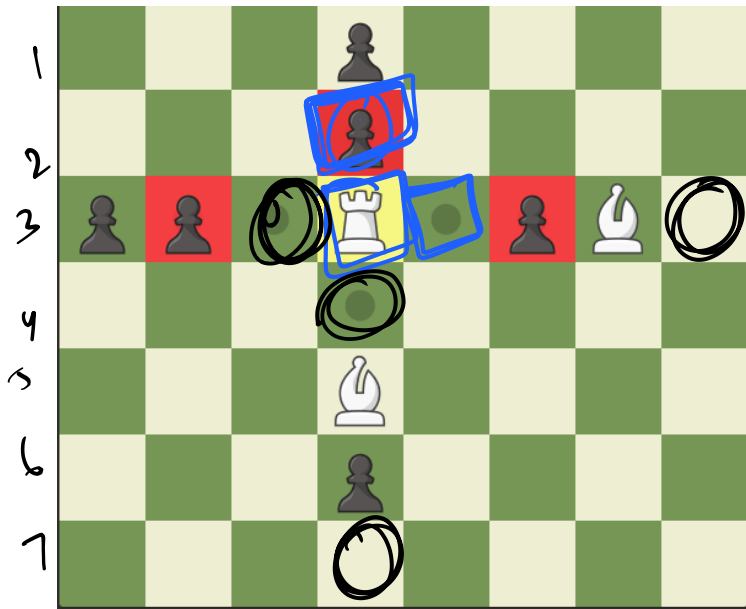
Note: A rook cannot move through other pieces, such as bishops or pawns. This means a rook cannot attack a pawn if there is another piece blocking the path.

Return the **number of pawns** the white rook is **attacking**.



`[i, j]`

row end



Directions	Common	Start	End
Top	j	$i-1$	0
Right	i	$j+1$	7
Down	j	$i+1$	7
Left	i	$j-1$	0

```

class Solution:
    def numRookCaptures(self, board: List[List[str]]) -> int:
        # Find rook
        pawns = 0
        for i in range(8) :
            for j in range(8) :
                if board[i][j] == 'R' :
                    # Top -> j common, i - 1 to 0
                    for k in range(i - 1, -1, -1) :
                        if board[k][j] == 'B' :
                            break
                        elif board[k][j] == 'p' :
                            pawns += 1
                            break
                    # Right -> i common, j + 1 to 7
                    for k in range(j + 1, 8) :
                        if board[i][k] == 'B' :
                            break
                        elif board[i][k] == 'p' :
                            pawns += 1
                            break

```

```

        # Down -> j common, i + 1 to 7
        for k in range(i + 1, 8) :
            if board[k][j] == 'B' :
                break
            elif board[k][j] == 'p' :
                pawns += 1
                break
        # Left -> i common, j - 1 to 0
        for k in range(j - 1, -1, -1) :
            if board[i][k] == 'B' :
                break
            elif board[i][k] == 'p' :
                pawns += 1
                break
        return pawns

```